ImageVault setup guide

How to open ImageVault, run it on your own computer, and put it on the web with Microsoft Azure.

I want to…	Go to	Time
Just use the website	Part A	1 minute
Run ImageVault on my laptop (no Azure needed)	Part B	15 minutes
Put my own copy on the web with Azure	Part C	30 minutes
Let my laptop do the AI work for the website	Part D	5 minutes
Fix a problem	Part E	

How to read the grey boxes. They are commands. Type them (or copy and paste them) into **PowerShell**, one line at a time, and press Enter after each line. To open PowerShell, press the Windows key, type **PowerShell** and press Enter.

Part A. Use the website

Nothing to install. It works on any computer or phone, on any Wi-Fi, including the college lab.

1 Open the website

Go to **https://imagevault-4g8kx.azurewebsites.net** in any browser.

The first visit of the day can take up to a minute: the free Azure plan sleeps when nobody uses it. Wait and it will load.

2 Log in

Type the website password. Ask the project owner for it; it is not written in this guide because the guide is public.

3 Add the demo photos

Download **https://imagevault-4g8kx.azurewebsites.net/demo-photos.zip**, right-click it and choose **Extract All**. Then drag all the photos from the folder onto the website.

Wait until the Home page stops saying Analysing…. Then look at Albums, People and Duplicates.

Starting a demo fresh: go to **Cloud** → **Storage** → **Delete everything**. It removes all photos, people and albums, so you can upload the demo set again.

Part B. Run it on your computer

This runs the whole website on your laptop. It doesn't use Azure at all: photos are kept in a folder and the data in a small file, both inside the project folder.

1 Install Python, Node.js and Git (once)

```
winget install -e --id Python.Python.3.12
winget install -e --id OpenJS.NodeJS.LTS
winget install -e --id Git.Git
```

Then **close PowerShell and open it again**, so it finds the new programs. Check that they work:

```
python --version
node --version
git --version
```

2 Download the code (once)

```
cd $HOME\Desktop
git clone https://github.com/Arnav-Dugad/imagevault-ai.git
cd imagevault-ai
```

This makes an **imagevault-ai** folder on your Desktop.

3 Set up Python (once)

```
python -m venv .venv
.venv\Scripts\activate
pip install -r requirements.txt
```

After the second line, **(.venv)** appears at the start of the line. That means it worked. The last line downloads the libraries and takes a few minutes.

4 Build the website (once, and after every code change)

```
cd frontend
npm install
npm run build
cd ..
```

5 Start ImageVault

```
python -m app
```

Open **http://127.0.0.1:8000** in your browser. There's no password when it runs on your computer.

The first start downloads the free AI models (about 225 MB, only once). The Home page shows when the AI is ready.

6 Stop it, and start it again next time

To stop: click on PowerShell and press **Ctrl+C**. Next time you only need:

```
cd $HOME\Desktop\imagevault-ai
.venv\Scripts\activate
python -m app
```

> **Error: "running scripts is disabled on this system"** when you activate? Run this once, then try again:
> `Set-ExecutionPolicy -Scope CurrentUser RemoteSigned`

Part C. Put it on the web with Azure

This creates your own copy of the website on Microsoft Azure. For this project it's already done: follow these steps only to set it up on a new Azure account. You need an **Azure for Students** account and a **GitHub** account, and your own copy (fork) of the repository on GitHub.

1 Install the cloud tools (once)

```
winget install -e --id Microsoft.AzureCLI
winget install -e --id Hashicorp.Terraform
winget install -e --id GitHub.cli
```

Close and reopen PowerShell afterwards. **az** talks to Azure, **terraform** creates the cloud resources, and **gh** talks to GitHub.

2 Sign in

```
az login
gh auth login
az account show --query id -o tsv
```

A browser opens for each sign-in. For **gh**, choose GitHub.com → HTTPS → Login with a web browser. The last command prints your **subscription ID**: copy it.

3 Fill in your settings

```
cd $HOME\Desktop\imagevault-ai\infra
copy terraform.tfvars.example terraform.tfvars
notepad terraform.tfvars
```

In Notepad, fill in three things and save: **subscription_id** (from step 2), **app_password** (the website password you want) and **alert_email** (where budget warnings go). This file stays on your laptop; it is never uploaded to GitHub.

4 Create everything on Azure

```
terraform init
terraform apply
```

Type **yes** when asked. It takes about 10 minutes. At the end it prints **website_url**: that's your website's address.

5 Connect GitHub to Azure (once)

```
terraform output -raw github_setup_commands
```

This prints four **gh** commands. Go back to the main folder (`cd ..`), paste them in and press Enter. Check with `gh secret list`: you should see three AZURE_ secrets.

6 Deploy the website

```
gh workflow run ci-cd.yml
gh run watch
```

GitHub tests the code and then puts it on Azure (about 5 minutes). Open your **website_url** and log in with your **app_password**. On the Cloud page, every service should say **Healthy**.

7 Updating the website later

Change the code, then save it to GitHub. Every push to **main** is tested and deployed automatically:

```
git add -A
git commit -m "Describe your change"
git push
```

Part D. Let your laptop do the AI work (optional)

Your laptop is about 10 times faster at recognising faces and photo contents than the free Azure plan. When the laptop is off, the website does the AI itself, so nothing breaks.

1 Start the worker

```
cd $HOME\Desktop\imagevault-ai
.venv\Scripts\activate
python -m app.worker
```

The first time, it asks for the **website address** and the **website password**, and offers to remember them in a file called **worker.env** (it is never uploaded to GitHub).

It prints a line for every photo it analyses, and the website's Cloud page shows **Laptop worker online**. Press **Ctrl+C** to stop.

2 Choose where the AI runs (optional)

On the website's **Cloud** page, the **Where the AI runs** switch has three settings:

Automatic: your laptop while the worker runs, otherwise Azure (the default).
My laptop: only your laptop; new photos wait until you start the worker.
Azure: only Azure; the laptop worker rests.

> **Works on any Wi-Fi.** The worker talks to the website over normal HTTPS, like your browser, so it works at home, in the college lab or on a phone hotspot. It needs nothing else from Azure.

Part E. If something goes wrong

The most common problems and how to fix them.

What you see	What to do
The website takes a long time to open	It's waking up (free plan). Wait one minute and refresh.
'python' or 'npm' is not recognized	Close PowerShell and open it again. If it still fails, install again (Part B, step 1).
'running scripts is disabled'	Run: Set-ExecutionPolicy -Scope CurrentUser RemoteSigned
The page says the website hasn't been built	Run Part B, step 4 (npm run build), then start again.
Address already in use (port 8000)	ImageVault is already running in another window. Close it, or open http://127.0.0.1:8000.
AI says 'Getting ready' for a long time	The models are downloading (225 MB, once). Check your internet.
terraform apply: region not allowed	Add location = "centralindia" to terraform.tfvars and run it again.
The deploy on GitHub was skipped	The GitHub secrets are missing: do Part C, step 5.
Laptop worker: 'Could not log in'	The password in worker.env is wrong. Delete worker.env and start the worker again.
Laptop worker: 'Can't reach the website'	Check your internet. The worker keeps trying on its own.

Costs, and turning it off

With the demo photos, Azure costs about **₹0 a month**: the website plan is free, the database is free for 12 months on a student account, and 1 GB of photos costs about ₹2 a month. The Cloud page shows this month's real cost in rupees, a forecast for the month, and a budget alert emails you at 50% and 80% of the US$5 budget.

When the course is over, delete everything on Azure so it can never use your credit:

```
cd $HOME\Desktop\imagevault-ai\infra
terraform destroy
```

Type **yes** when asked. Your code on GitHub and on your laptop is not touched.